

Reporte de Ejecución: Serialización y Deserialización de Objetos en Java

Proyecto: RetroGaming

1. Introducción

El presente reporte demuestra la correcta implementación de la persistencia de datos en Java mediante el uso de las clases `ObjectOutputStream` y `ObjectInputStream`. Se evidencia la escritura de un objeto principal de tipo inventario (el cual almacena en un `ArrayList` múltiples instancias de la clase videojuego) en un archivo físico, y su posterior recuperación en memoria de forma exitosa, resolviendo los conflictos de conversión de tipos (`ClassCastException`).

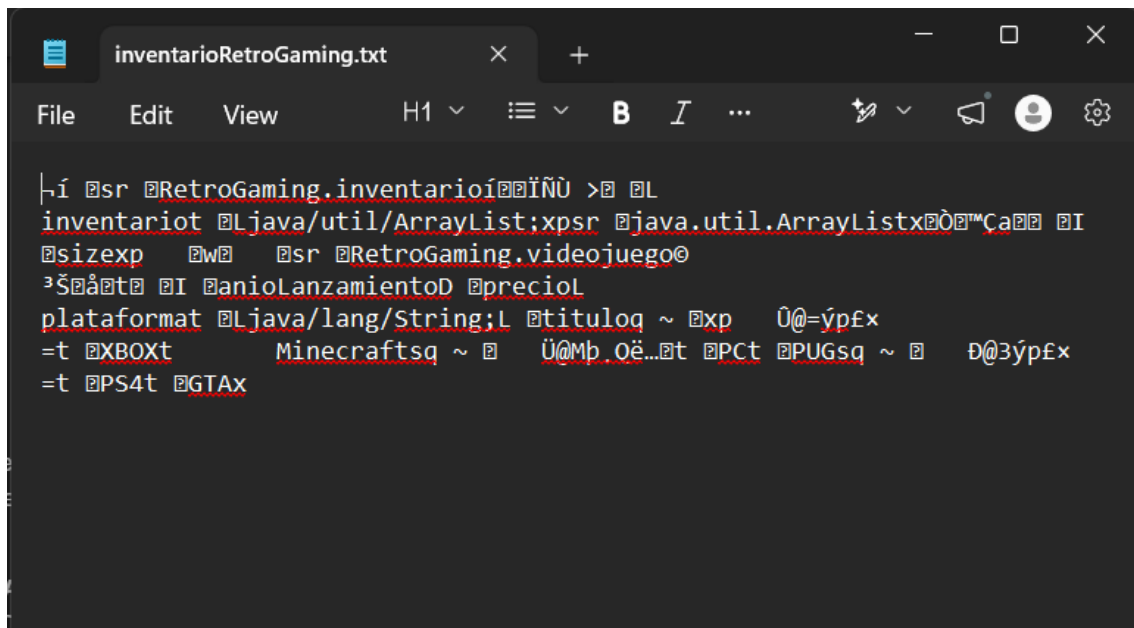
2. Proceso de Guardado de Datos (Escritura)

En esta fase inicial, la aplicación instancia los videojuegos, los registra en el inventario y utiliza el método de la clase `EscrituraObjetos` para serializar la estructura de datos completa.

```
run:
Titulo: Minecraft
Plataforma: XBOX
Año Lanzamiento: 2011
Precio: 29.99
Titulo: PUG
Plataforma: PC
Año Lanzamiento: 2012
Precio: 59.99
Titulo: GTA
Plataforma: PS4
Año Lanzamiento: 2000
Precio: 19.99
Inventario Guardado Exitosamente
```

3. Generación del Archivo Físico

Tras la ejecución del proceso de escritura, el flujo de datos (`FileOutputStream`) genera el archivo correspondiente en el almacenamiento local, conteniendo el estado de los objetos.



```
import java.util.ArrayList;
import java.util.Scanner;

public class RetroGaming {
    static ArrayList<Videojuego> inventario = new ArrayList<>();

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (sc.hasNext()) {
            String titulo = sc.nextLine();
            String plataforma = sc.nextLine();
            double precio = sc.nextDouble();
            int año = sc.nextInt();
            Videojuego vj = new Videojuego(titulo, plataforma, precio, año);
            inventario.add(vj);
        }
    }
}

class Videojuego {
    String titulo;
    String plataforma;
    double precio;
    int año;

    public Videojuego(String titulo, String plataforma, double precio, int año) {
        this.titulo = titulo;
        this.plataforma = plataforma;
        this.precio = precio;
        this.año = año;
    }
}
```

4. Proceso de Carga y Recuperación de Datos (Lectura)

Para comprobar la integridad de la serialización, la clase `LecturaObjetos` lee el archivo generado, deserializa el flujo de bytes y reconstruye el objeto inventario original en una nueva variable. Posteriormente, se imprimen los datos recuperados en consola.

```

public class Tienda1 {
    public static void main(String[] args) throws IOException {
        inventario inventarioJuegos = new inventario();
        String nombreArchivoInventario = "inventarioRetroGaming.txt";
        videojuego juego = new videojuego("Minecraft", "XBOX", 2011, 29.99);
        videojuego juego1 = new videojuego("PUG", "PC", 2012, 59.99);
        videojuego juego2 = new videojuego("GTA", "PS4", 2000, 19.99);

        inventarioJuegos.agregarVideojuego(juego);
        inventarioJuegos.agregarVideojuego(juego1);
        inventarioJuegos.agregarVideojuego(juego2);

        inventarioJuegos.listarInventario();

        escrituraObjetos escritura = new escrituraObjetos(nombreArchivoInventario, inventarioJuegos);
        escritura.escribirInventario();

        inventario juegosArchivo = new inventario();

        System.out.println("\nLectura de Archivo\n");

        lecturaObjetos lectura = new lecturaObjetos(nombreArchivoInventario);

        juegosArchivo = lectura.cargarLista();

        juegosArchivo.listarInventario();
    }
}

```

Lectura de Archivo

Archivo Leído Exitosamente

Título: Minecraft

Plataforma: XBOX

Año Lanzamiento: 2011

Precio: 29.99

Título: PUG

Plataforma: PC

Año Lanzamiento: 2012

Precio: 59.99

Título: GTA

Plataforma: PS4

Año Lanzamiento: 2000

Precio: 19.99

BUILD SUCCESSFUL (total time: 0 seconds)

5. Conclusión

Se logró implementar la persistencia de datos de manera estable. Al ajustar la lógica para serializar el objeto contenedor (inventario) de una sola vez, en lugar de iterar sobre cada elemento interno individualmente, se optimizó el flujo de Entrada/Salida (I/O) y se garantizó que el casteo de objetos durante la lectura coincidiera exactamente con la estructura de datos almacenada.